IEEE Open Access Journal of
Power and Energy

Received 10 November 2020; revised 20 March 2021; accepted 19 April 2021.
Date of publication 3 May 2021; date of current version 19 May 2021.

Digital Object Identifier 10.1109/OAJPE.2021.3077218

Fully Decentralized Reinforcement Learning-Based Control of Photovoltaics in Distribution Grids for Joint Provision of Real and Reactive Power

RAYAN EL HELOU (Student Member, IEEE), DILEEP KALATHIL (Senior Member, IEEE), AND LE XIE (Senior Member, IEEE)

Department of Electrical and Computer Engineering, Texas A&M University, College Station, TX 77843 USA

CORRESPONDING AUTHOR: L. XIE (le.xie@tamu.edu)

This work was supported in part by the NSF under Grant CRII-CPS-1850206, Grant CAREER-EPCN-2045783, Grant ECCS-1839616, and Grant ECCS-2038963; and in part by the Department of Energy under Contract DE-EE0009031.

ABSTRACT In this paper, we introduce a new framework to address the problem of voltage regulation in unbalanced distribution grids with deep photovoltaic penetration. In this framework, both real and reactive power setpoints are explicitly controlled at each solar panel smart inverter, and the objective is to simultaneously minimize system-wide voltage deviation and maximize solar power output. We formulate the problem as a Markov decision process with continuous action spaces and use proximal policy optimization, a reinforcement learning-based approach, to solve it, without the need for any forecast or explicit knowledge of network topology or line parameters. By representing the system in a quasi-steady state manner, and by carefully formulating the Markov decision process, we reduce the complexity of the problem and allow for fully decentralized (communication-free) policies, all of which make the trained policies much more practical and interpretable. Numerical simulations on a 240-node unbalanced distribution grid, based on a real network in Midwest U.S., are used to validate the proposed framework and reinforcement learning approach.

INDEX TERMS Unbalanced distribution grids, photovoltaic inverters, voltage regulation, reinforcement learning.

I. INTRODUCTION

PHOTOVOLTAIC (PV) smart inverter technology introduced in recent years enables solar panels to act as distributed energy resources (DERs) that can provide bi-directional reactive power support to electric power grid operations [1]–[3]. This support can be used to regulate local and system-wide voltages in distributed grids, and the IEEE Standard 1547-2018 [4] provides requirements on the use of such support. Voltage regulation is critical for network safety, both at the transmission and distribution levels.

In the distribution grid, voltage regulation is usually controlled either through discrete switching (e.g. tap transformers, capacitor banks) or devices with continuous set points (e.g. PV inverters). Broadly speaking there are two categories of control and information structure to address the voltage regulation in distribution grids. The first category of solutions assumes complete or partial knowledge of system parameters and topology (such as a sample of references [5]–[11]). The second category of solutions primarily relies on observation data and do not explicitly use the knowledge of a physical model of the distribution network (such as a subset of references [12]–[18]).

In the first set of solutions, control schemes are adopted based on assumed system models and on knowledge of line parameters. Given the broad set of references, we provide representative ones to illustrate some key directions of research. In [5], a multi-objective OPF problem is solved over a given unbalanced distribution grid model using Sequential Quadratic Programming (SQP), where the set of objectives include minimizing line losses, voltage deviation from nominal, voltage phase unbalance and power generation and curtailment costs. In [6], a model-based voltage regulation problem is shown to be solvable by an equivalent Semi-Definite Programming (SDP) problem, and the

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

sufficient conditions under which it can be solved as a result of convexifying the problem are outlined. In [7]–[9], the distribution grid is modelled using the widely adopted linearized flow model, known as LinDistFlow, that assumes a tree (radial) grid structure and negligible line losses. In these papers, the same voltage regulation problem is solved, and an extension to these works are [10], [11] in which limited or no communication between buses is needed and the same LinDistFlow model is adopted to provide theoretical guarantees on convergence and stability of the proposed control schemes.

In the second set of solutions, reinforcement learning (RL) approaches are used to bypass the need to *explicitly* model the system. Nonetheless, some form of a system model and a power flow solver is still needed to simulate the effects of actions on states, as it may not be feasible to learn by directly interacting with a live power grid. However, knowledge of such a model or its parameters need not be *explicitly* known by the solver of the RL problem, since optimal policies can be inferred solely by interacting with the simulation environment and receiving reward signals. This flexibility to system models is in part what makes RL approaches attractive, in contrast with the aforementioned approaches that rely heavily on a very specific class of system models. Furthermore, the set of objective functions commonly used in conventional control methods tend to be restrictive (e.g. quadratic), whereas in RL-based approaches, reward functions can be arbitrary and are usually designed to more directly reflect the user's underlying objective.

In [12], for example, Batch RL is adopted to solve the optimal setting of voltage regulation transformers, where a virtual transitions generator is used to allow the RL agent to collect close-to-real samples, for learning, without jeopardizing real-time operation. In [13], the Optimal Reactive Power Dispatch (ORPD) problem is solved using tabular Q-learning, where the objective is to minimize line losses, and actions include various forms of discrete reactive power redispatch, including switching control of tap transformers and of capacitor banks. Tabular Q-learning works well when there is a relatively small number of discrete states and actions, but suffers from the *curse of dimensionality* and does not scale up well for larger systems. In [14], *deep* RL is used to optimize reactive power support over two timescales: one for discrete capacitor configuration and the other for continuous inverter setpoints. Here *deep* refers to the representation of Q-value functions (used in Q-learning) in deep neural network form, as opposed to tabular form. This approach is well known as DQN (Deep Q-learning). This is also applied in [15] to control voltages on the transmission level. While DQN uses neural networks to approximate value functions over continuous state spaces, it still relies on the assumption that the action space is discrete. Policy gradients, which we review later in this paper, are an alternative set of RL approaches that enable continuous action spaces, and in [16]–[18], policy gradients are used to solve Volt-VAR control problems where reactive power support is selected from a continuous set using a deep neural network that maps states directly to actions.

Such methods are inherently limited by physical constraints on reactive power support which are conventionally assumed to be uncontrollable. Here, we discuss the flexibility of such constraints and the value of relaxing them. For example, conventional practices of maximum power-point tracking (MPPT) have been state-of-the-art, wherein each PV inverter is designed to extract the maximum real/active power from the solar panel. However, with a growing number of PV panels in the distribution grid, it becomes important to fully investigate the benefits and costs of always absorbing the maximum real power from the sun into the grid in real-time. By absorbing less real power, for instance, there is more room for reactive power support. In practice, real power curtailment at generators is performed either by solving model-based OPF problems, such as in [5], or by using a so-called Volt-Watt (VW) droop controller, which requires carefully tuning the droop curves at each generator.

In this paper, an RL approach is proposed for directly learning decentralized joint active and reactive power support. We illustrate a set of scenarios where instead of injecting all of the solar power into the network, it might rather be better to save or store the power and to inject it at a later time. Even in the absence of a storage system, under deep enough photovoltaic penetration, we discover that the RL agent surprisingly learns by itself that it might be better to draw only parts of the available power, in order to avoid over-voltage, especially if there is an insufficient amount of reactive power resources available. This is accomplished by designing a reward function, used in the RL approach, that strikes a balance between voltage regulation and real power absorption.

Despite modern advancements in deep RL, and in artificial intelligence and machine learning more broadly, the adoption rate of such tools in the power sector is still substantially low compared to other sectors. This is primarily due to the difficulty in interpreting deep neural networks, and due to the complexity of setting up processes that are required for the use of such tools in practice. These issues make it quite risky for a power grid operator to both trust those tools and to prefer them over conventional ones for operating over critical infrastructure. It is thus necessary for the adoption of RL by grid operators that the proposed approaches are heavily simplified and made much more interpretable, which is what we aim to contribute in this study.

The key contributions of this paper are suggested as follows:

1) Joint optimization of real and reactive power injection from the PV generation is formulated, with a parameterized reward function tailored for a multi-agent RL approach. Such parameterization is shown to facilitate more interpretable and easier to train deep RL policies, for a more user-friendly experience.
2) A variant of PPO (Proximal Policy Optimization), a popular RL approach for handling continuous action

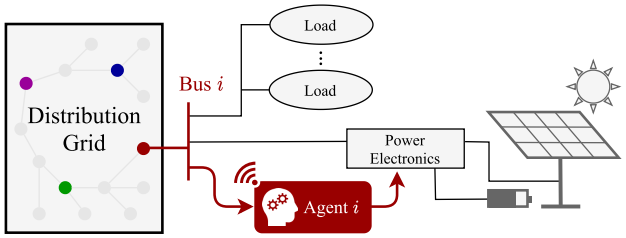


FIGURE 1. Every agent in the distribution grid controls a local photovoltaic system, with possible communication between agents. Note: each colored dot in the "Distribution Grid" box refers to one agent.

spaces, is proposed for *decentralized* settings, to dramatically simplify the search for optimal policies for relatively large systems. Not only can it be shown to train as well as or better than a centralized policy architecture, but it is much easier to implement than RL-based decentralized approaches in the existing body of literature.

In our framework, an RL agent per bus observes voltages locally and incrementally updates real and reactive power setpoints at photovoltaic inverters, similar to an integral droop controller (e.g. [19]). However, it does not rely on any explicit knowledge of network topology or line parameters, and is fully decentralized, requiring minimal communication infrastructures for practical implementation. This is illustrated in Fig. 1, wherein agents are shown to be able to communicate with other agents or a central controller. The word *fully* in the title of this paper refers to the fact that under our proposed framework, each agent in real-time (online) executes its policy locally without communicating with any other agent, not even any of its neighbors. During training though (while exploring and updating policies), observations are aggregated centrally so that all agents can adapt to one another. Since this is done on a much longer-term basis, the communication technologies used do not need to be as sophisticated for this purpose as they might be for an alternative online communication-based approach.

Finally, the term "load" in Fig. 1 is not restrictive. For example, it can include individual residential units or larger and more aggregate neighborhoods. In the former case, examples include houses with rooftop solar and the agent is controlling a few panels, and in the latter case, one bus could correspond to a solar farm and the agent would control many units simultaneously.

The remainder of the paper is organized as follows. In Section II, the voltage regulation objective with joint real and reactive power compensation is formulated. In Section III, we provide a general review of Markov Decision Processes, and one specific to our problem in Section IV, with modifications to simplify the task. In Section V, centralized and decentralized policy architectures are proposed, and they are evaluated in Section VI with numerical simulations.

II. PRELIMINARIES

We consider a three-phase balanced distribution network which consists of a set $\mathcal{N} = \{0, 1, \ldots, N\}$ of buses and a set $\mathcal{L} \subset \mathcal{N} \times \mathcal{N}$ of distribution lines connecting the buses. Bus 0 represents the substation that acts as the single point of connection to a bulk power grid. For each distribution line $(i, j) \in \mathcal{L}$, $y_{i,j}$ denotes its admittance.

The set of algebraic power flow equations that govern this three-phase balanced (single-phase equivalent) network are:

$$P_i - \mathbf{i}Q_i = \widetilde{V}_i^* \sum_{j \in \mathcal{N}} y_{ij} \widetilde{V}_j, \quad \forall i \in \mathcal{N}, \tag{1}$$

where P_i and Q_i are the net injection of real and reactive power, $\widetilde{V}_i$ is the complex phasor voltage at bus i. $\widetilde{V}_i^*$ denotes the complex conjugate of $\widetilde{V}_i$ and $\mathbf{i} := \sqrt{-1}$.

To model a distribution grid that is not three-phase balanced, or *unbalanced* for short, you may simply replace bus indices with phase indices in Eq. (1), and $\mathcal{N}$ with the set of all phases. With this, you can generalize over two-phase and single-phase buses, which are common in real distribution grids.

Let V_i denotes the *positive sequence* voltage magnitude at bus i. At the substation bus, V_0 is fixed at 1.0 p.u. as it is modeled as an ideal voltage source.

Let $\mathcal{C} \subset \mathcal{N}$ be the set of buses that are equipped with solar panels and controllable smart inverters, and $n := |\mathcal{C}|$. Let P_i^c and Q_i^c be the total real and reactive power, respectively, injected by the PV inverter at bus $i \in \mathcal{C}$. Each inverter has an apparent power capacity S_i, which limits P_i^c and Q_i^c as

$$(P_i^c)^2 + (Q_i^c)^2 \leq S_i^2, \quad P_i^c \leq p^{\text{env}} \leq 0.9 S_i, \tag{2}$$

where p_i^{env} is the maximum amount of real power that can be drawn from the solar panel at a given moment in time. It depends on exogenous *environmental* factors (irradiance, temperature, etc.), hence the superscript. The upper bound on this quantity is $0.9S_i$ since each inverter in the network is assumed to obey standard IEEE 1547-2018 [4]. We let this injected power be evenly distributed across all phases per bus.

Strictly speaking, if $P_i^c(t)$ is the actual real power injected by the inverter at time t, and $\overline{P_i^c}(t)$ is the setpoint, then those two cannot be equal at the same time. There is a small time delay ($\sim$ 10 ms, or less than one 60 Hz cycle) between when the setpoint is assigned and when the actual quantity tracks it. We let both the discrete time step and the tracking time be 10 ms. This allows us to treat the system as a quasi-steady state system, as illustrated in Fig. 2.

More precisely, net real and reactive power injections at bus i at time t can be expressed as $P_i(t) = P_i^c(t) - P_i^l(t)$ and $Q_i(t) = Q_i^c(t) - Q_i^l(t)$, where $P_i^l(t)$ and $Q_i^l(t)$ are the (uncontrollable) load consumption at bus i at time t. Due to the one time step delay between setpoint and actual injections, $P^c(t+1) = \overline{P^c}(t)$ and $Q^c(t+1) = \overline{Q^c}(t)$. Then, due to the quasi-steady state nature, $V(t+1) = f(\overline{P^c}(t), \overline{Q^c}(t))$, where $f(\cdot)$ represents the solution to the algebraic power flow equation given in (1).

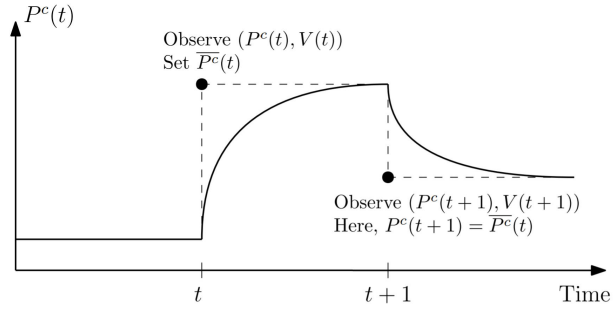


FIGURE 2. **Illustration of quasi-steady state behavior.**

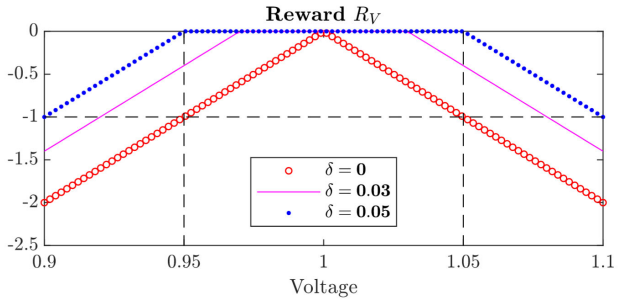


FIGURE 3. **Voltage deviation reward function R_V for different δ (see Eq. (3f)).**

We address the problem of voltage regulation in the distribution grid through joint real and reactive power control of PV inverter setpoints. The control objective is to track desired voltage levels while not wasting solar power in the process. The voltage regulation problem can be formulated as follows:

$$\underset{\overline{P^c}, \overline{Q^c}}{\text{maximize}}\ \mathbb{E}\left[\sum_{t=0}^{T}\sum_{i\in\mathcal{C}} R_{V_i(t)} + \mu_i R_{P_i^c(t)}\right], \tag{3a}$$

$$\text{s.t. } P_i(t) = P_i^c(t) - P_i^l(t), \tag{3b}$$

$$Q_i(t) = Q_i^c(t) - Q_i^l(t), \tag{3c}$$

$$\text{Eq. (1), i.e. } \textit{power flow}, \forall t, \tag{3d}$$

$$\text{Eq. (2), i.e. } \textit{inverter constraints}, \forall t, \tag{3e}$$

$$R_{V_i(t)} = \frac{1}{0.05}\min\left\{\delta - |1 - V_i(t)|, 0\right\}, \tag{3f}$$

$$R_{P_i^c(t)} = \frac{P_i^c(t)}{0.9S_i}, \tag{3g}$$

where μ_i's and δ are positive scalars. Unlike with P^c and Q^c (controllable inverter), P^l and Q^l (uncontrollable load) are generally not evenly distributed across all individual phases. For clarity, subscript i refers to i^{th} bus index, and terms where the subscript is dropped refer to all buses collectively.

Voltage deviation (from nominal 1.0 p.u.) at each bus is considered acceptable if it is kept within some user-defined δ. Deviations greater than this are assigned negative rewards, as depicted in Fig. 3, to signify an undesirable voltage profile. The reward term R_V in (3f) quantifies this criteria.

In conventional voltage regulation schemes, reactive power injection or consumption is used to alleviate voltage deviation problems. Reactive power support is limited by physical constraints. For example in the case of PV inverters, as expressed in Eq. (2), real power injection by the solar panel directly limits available reactive power. In our proposed objective, we not only consider reactive power but real power provision as well as a decision variable to add a degree of freedom and to relax constraints on reactive power support if needed. Since we seek to extract as much real power as possible from the solar panel, physically bounded by $0.9S_i$ as expressed in Eq. (2), we assign a positive reward to more power drawn from the solar panels, as quantified in (3g) and (3).

Under this framework with joint provision of real and reactive power, the user (e.g. utility) selects parameter μ. This parameter acts as a balancing term, between voltage deviation minimization and solar production maximization, considering the fact that over-injection of power leads directly both to over-voltage and to tighter constraints on reactive power. A value for μ chosen too high yields a control scheme equivalent to conventional maximum power point tracking (MPPT) control with limited reactive power support. On the other hand, as μ approaches zero, much less real power is likely to be drawn from solar panels by the optimal controller. In section VI, we demonstrate a balanced choice of μ.

The following assumptions are made about variables that are not explicitly controlled:

- While provided to the simulator, neither network topology nor line parameters are used by the controller at any time during training or execution. That is, no a priori knowledge of such values is needed.
- No load or solar forecasting is made available to the controller, neither upon training nor during execution.
- Net load at a control bus is measured by the controller before supplying a setpoint to the solar panel inverter.

The voltage regulation problem formulated in this section is posed as a Markov Decision Process (MDP) in Section IV, but first, in Section III, MDP terminology and formalism is introduced for a more general class of control problems.

III. MARKOV DECISION PROCESS AND REINFORCEMENT LEARNING

In this section, we give a brief review of the Markov Decision Processes (MDP) terminology. We will also review a specific RL algorithm, called Proximal Policy Optimization (PPO) [20], which we use to solve the voltage regulation problem.

MDP is a standard formalism for modeling and solving control problems. The goal is to solve sequential decision making (control) problems where the control actions can influence the evolution of the state of the system. An MDP can be defined as a four-tuple $(\mathcal{S}, \mathcal{A}, \bar{P}, R)$, where $\mathcal{S}$ is the state space and $\mathcal{A}$ is the action space. $\bar{P}(s'|s, a)$ is the probability of transitioning from state s to s' upon taking action a, and $R(s)$ is the reward collected at this transition. We consider a finite horizon MDP setting with horizon (episode) length of T. We assume that the rewards depend only on the state,

not on the actions. A control policy $\pi : \mathcal{S} \to \mathcal{A}$ specifies the control action to take in each possible state. The performance of a policy is measured using the metric of value of a policy, $\bar{V}_\pi$, defined as,

$$\bar{V}_\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} R(s(t)) | s(0) = s \right], \tag{4}$$

where, $s(t+1) \sim \bar{P}(\cdot|s(t), a(t)), a(t) = \pi(s(t))$, and $s(t)$ is the state of the system at time t and $a(t)$ is the action taken at time t. The goal is to find the optimal policy π^* that achieves the maximum value, i.e, $\pi^* = \arg\max_\pi \bar{V}_\pi$. The corresponding value function, $\bar{V}^* = \bar{V}_{\pi^*}$, is called the optimal value function. π^* and $\bar{V}^*$ satisfy the Bellman equation,

$$\pi^*(s) = \underset{a \in \mathcal{A}}{\operatorname{argmax}} \left[R(s) + \sum_{s' \in \mathcal{S}} \bar{P}(s'|s, a) \bar{V}^*(s') \right] \tag{5}$$

When the system model $\bar{P}$ is known, the optimal policy and value function can be computed using dynamic programming [21]. However, in most real-world applications, the system model is either unknown or extremely difficult to model. In the voltage regulation problem, the line parameters and/or the topology of the network may not be known a priori. Even if they were known, it would still be difficult to model the effect of actions on states in a feed-forward fashion, due to the algebraic nature of their relationship. In such scenarios, the optimal policy has to be learned from sequential state/reward observations by interacting with an environment, which in this paper is a simulation environment as described in Section IV. Reinforcement learning is the approach for computing the optimal policy for an MDP when the model is unknown.

Policy gradient algorithms are a popular class of RL algorithms. In a policy gradient algorithm, we represent the policy as π_θ, where θ denotes parameters of the neural network used to represent the policy. Let $J(\theta) = \mathbb{E}_s[\bar{V}_{\pi_\theta}(s)]$, where the expectation is w.r.t. to a given initial state distribution. The goal is to find the optimal parameter $\theta^* = \arg\max_\theta J(\theta)$. This is achieved by implementing a gradient descent update, $\theta_{k+1} = \theta_k + \alpha_k \nabla J(\theta_k)$, where α_k is the learning rate. The gradient $\nabla J(\theta)$ is given by the celebrated policy gradient theorem [21] as $\nabla J(\theta) = \mathbb{E}_{\pi_\theta}[\bar{Q}_{\pi_\theta}(s, a) \nabla \log \pi_\theta(s, a)]$, where expectation is w.r.t. the state and action distribution realized by following the policy π_θ. Here $\bar{Q}_{\pi_\theta}$ is the Q-value function corresponding to the policy π_θ. Often, the Q-value function is represented using a neural network of its own (different from the one used for policy representation). The neural network which represents the policy is called the actor network and that which represents the value function is called the critic network. The terminology is due to the fact that the policy network determines actions given observations and the value network provides the 'critic' feedback to update the policy parameter, as clear from the expression for $\nabla J(\theta)$. This class of algorithms is also called actor-critic algorithms. The goal is to incrementally update the parameters of both networks in such a way that they converge to yield parameters corresponding to optimal policy and Q-value functions.

Trust region policy optimization (TRPO) [22] is a recent variant of policy gradient algorithms. For improving the sample efficiency and ensuring reliable convergence, TRPO modifies the policy update as

$$\theta_{k+1} = \underset{\theta}{\operatorname{argmax}}\, \mathbb{E}_{\theta_k} \left[\frac{\pi_\theta(s, a)}{\pi_{\theta_k}(s, a)} \bar{Q}_{\pi_{\theta_k}}(s, a) \right] \tag{6}$$

$$\text{s.t. } \mathbb{E}\left[D_{\mathrm{KL}}(\pi_{\theta_k}(s, \cdot), \pi_\theta(s, \cdot)) \right] \leq d, \tag{7}$$

where, $D_{\mathrm{KL}}(\cdot, \cdot)$ is the Kullback-Leibler divergence between two policies, and $\pi_\theta(s, a)$ denotes probability of selecting action a given state s. Constant d is a user-defined threshold.

Proximal policy optimization (PPO) algorithm [20] builds upon the TRPO framework by modifying the objective function and optimization update which enables improved data efficiency and easier implementation since the KL divergence constraint is dropped and the objective is clipped (or saturated) as described in [20]. Clipping of the objective is performed to discourage the optimizer from over-updating θ. We adapt this state-of-the-art algorithm to a decentralized setting to solve the voltage regulation problem we consider.

IV. VOLTAGE REGULATION AS AN RL PROBLEM

We use the MDP formalism to model the voltage regulation problem presented in Section II. As a reminder, $n := |\mathcal{C}|$.

1) STATE SPACE

The state space $\mathcal{S} \subset \mathbb{R}^{2n}$ is the set of real power injections and voltage measurement at all controllable buses. For convenience, each state $s \in \mathcal{S}$ is defined as an affine transformation of those measurements. More precisely, the state of the system at time t, $s(t)$, is given by

$$s(t) := (s_1^P(t),\ s_1^V(t),\ \cdots,\ s_n^P(t),\ s_n^V(t)),$$
$$\text{where, } s_i^P(t) \leftarrow \frac{P_i^c(t)}{0.9 S_i} - 1, \quad s_i^V(t) \leftarrow \frac{1 - V_i(t)}{0.05}. \tag{8}$$

So, when $P_i^c(t)$ is its maximum allowable value, $s_i^P(t)$ is zero. Also, when the voltage is equal to the nominal value, $s_i^V(t)$ is zero. Thus, ideal scenarios correspond to the state value zero, and critical scenarios correspond to magnitudes of order one or less, assuming critical voltages exceed 1 ± 0.05. This scaling helps to initialize and train the RL algorithm.

2) ACTION SPACE

Given voltage measurements at every bus in $\mathcal{C}$, we can choose two approaches to determine $\overline{P^c}$ and $\overline{Q^c}$: 1) Directly determine optimal real and reactive power setpoints, i.e. $\left(\overline{P^c}, \overline{Q^c}\right)$, by algebraically tying to voltage, or 2) change setpoints incrementally, i.e. $\left(\Delta\overline{P^c}, \Delta\overline{Q^c}\right)$, similar to an integral controller. The first approach requires the design and memorization of a highly non-linear function that is likely dependent on system operating conditions. Due to the lack of tracking in this approach, forecasting would be required to respond to

different operating conditions. The second approach, on the other hand, enables tracking a desired state in a simple and incremental way [11]. We use the second approach in this paper.

The action space $\mathcal{A} \in [-1, 1]^{2n}$ is the set of possible *scaled increments* in real and reactive power setpoints. The action at time t, $a(t)$, is given by $a(t) = (a_1^P, a_1^Q, \cdots, a_n^P, a_n^Q)$ and the increment to those setpoints are defined respectively as $\Delta\overline{P_i^c} = a_i^P \cdot \Delta_{\max}^P$ and $\Delta\overline{Q_i^c} = a_i^Q \cdot \Delta_{\max}^Q$. Here $\Delta_{\max}$ explicitly limit the size of actual (as opposed to scaled) increments $\left(\Delta\overline{P^c}, \Delta\overline{Q^c}\right)$.

3) TRANSITION MODEL

We assume that *next states* are obtained by interaction either with a real-world distribution grid or with a simulator, such as OpenDSS [23]. In the case of a simulator, provided changes in loads $(P^l,\ Q^l)$ and current state and action based on $\mathcal{S}$ and $\mathcal{A}$, next states can be computed directly. For example, actions are mapped to states using OpenDSS as follows:

$$V(t+1) = \text{OpenDSS}(\overline{P^c}(t), \overline{Q^c}(t)) \tag{9}$$

We choose to use OpenDSS simulator for two main reasons: (a) It can solve power flow for unbalanced distribution grids, and (b) one can directly interact with it using Python, where RL methods are easier to implement.

4) REWARD FUNCTION

System-wide reward at every time step is obtained as follows:

$$R_t = \frac{1}{n}\sum_{i\in\mathcal{C}} R_{V_i(t)} + \mu_i R_{P_i^c(t)} \tag{10}$$

where $R_{V_i(t)}$ and $R_{P_i^c(t)}$ are defined in Eq. (3g,3f).

Note that the state and action spaces have been defined in such a way that each element ranges from -1 to 1, with an exception where the voltage-related state may exceed ± 1 if the p.u. voltage exceeds 1 ± 0.05 under abnormal conditions. This is a suitable choice for training an RL policy as it allows for initialization and adjustment of policy parameters θ in a standard way by exploiting the state of the art algorithms (most of which requires that state and action spaces be a *box* inside ± 1 along all dimensions).

Based on the definition of action space, the RL agent seeks to learn the *magnitude and direction* in which to incrementally change the setpoints, for every starting state. This raises the question: what information does the agent need to guide this action? The state defined in Eq. (8) has the following advantage: If both the voltage term and the power term are zero (i.e. maximum power drawn and nominal voltage), then the scenario is ideal and no *extra* injection is needed. If the load changes in the system, though, a simple amendment to the RL controller is needed:

$$\begin{aligned} \Delta\overline{P^c} &\leftarrow \Delta_{max}^P \cdot a_P + \Delta P^l \\ \Delta\overline{Q^c} &\leftarrow \Delta_{max}^Q \cdot a_Q + \Delta Q^l \end{aligned} \tag{11}$$

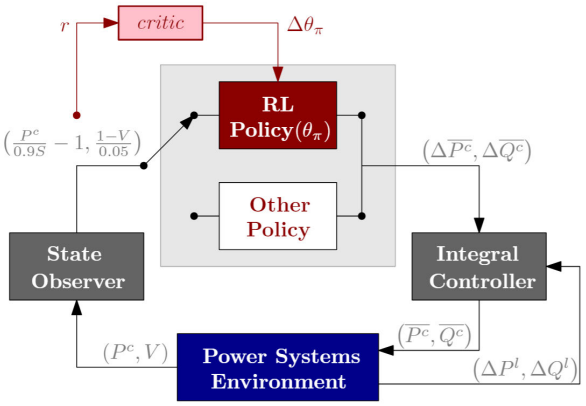


FIGURE 4. RL-based Control Architecture.

where a_P and a_Q (both in $[-1, +1]$) are determined by the RL agent's zero-centered policy π, and $\left(\Delta P^l, \Delta Q^l\right)$ is the observed change in load at the controllable buses.

The strategy adopted in Eq. (11) is termed an integral controller since setpoint $(\overline{P^c}, \overline{Q^c})$ behaves as a discrete-time integrator of changes in operating conditions. Moreover, this controller tracks the state to zero in steady state, within resource limits, since all terms in Eq. (11) go to zero if $s = 0$. Under scarcity of resources, one or more of the state terms in Eq. (8) will be non-zero, which calls for a balance between *maximum power point tracking* and *voltage regulation*.

Note that state-tracking incremental setpoint changes are bounded by Δ_{max}^P and Δ_{max}^Q to limit fluctuations. These values are chosen heuristically as $0.09S_i$ and $0.2S_i$ respectively since those are one-tenth of the maximum possible jumps in setpoints $\overline{P^c}$ and $\overline{Q^c}$.

V. CONTROL POLICY ARCHITECTURE AND OPTIMIZATION

In this section, we present the design and architecture of our RL algorithm for voltage regulation. We build on the PPO [20] algorithm and extend it to a decentralized setting. Fig. 4 summarizes the RL-based control policy architecture we propose. *State Observer* refers to Eq. (8) and *Integral Controller* refers to Eq. (11). $\Delta\theta_\pi$ refers to changes in policy π determined by the PPO algorithm. The policy and critic network architectures used to implement PPO are detailed in the remainder of this section.

The standard PPO algorithm assumes that there is a single centralized agent which fully observes the state of the system and can take any control action. However, this centralized control policy may not be feasible for the voltage regulation problem we consider. Firstly, the RL algorithm may not be able to scale to a large network with many nodes. Secondly, even if a centralized training of an RL algorithm is feasible, the implementation of such a centralized control policy in the real-world system may not be possible due to the communication infrastructure needed. We propose a decentralized RL algorithm that overcomes these challenges.

Recall that s_i and a_i refer local state and action (at bus i) respectively. As defined in Eq. (8), each state s_i contains two terms per bus, relating to real power and voltage measurements. Similarly, action a_i was defined in such a way that it also contains two terms per bus, relating to changes in real and reactive power setpoints. Our goal is to find the optimal policy parameter θ_i^* for each bus i that maps local state $s_i(t)$ to local control action $a_i(t)$, i.e. $a_i(t) = \pi_{\theta_i^*}(s_i(t))$, in an optimal way. The objective is to maximize the cumulative global (system-wide) reward.

To replace the centralized policy, used in standard PPO, with a decentralized policy, we propose a neural network architecture for π that connects input to output only at the same bus, rendering it equivalent to a decentralized controller, to compete with conventional methods. That is, there are n neural networks in parallel, each with just 2 inputs and 2 outputs. Each of those smaller networks is parametrized by a group of weights and biases, denoted collectively as θ_i, and notation π_{θ_i} is shortened to π_i. This time, the PPO algorithm optimizes over $\theta_1, \ldots, \theta_n$, in search of optimal policies $\pi_1, \ldots, \pi_n$, where

$$a_i \leftarrow \pi_i(s_i) \quad \forall i \in \mathcal{C} \tag{12}$$

Note that the only difference between this case and the centralized case (optimizing over θ), is that here we enforce the strict rule that all neural network weights connecting states at bus i to actions at bus j are fixed at zero iff $i \neq j$. One can also modify this architecture by replacing the condition $i \neq j$ with $(i, j) \notin \mathcal{L}$, if the desired setup involves neighboring buses communicating with one another. For training and implementation purposes, we perform orthogonal initialization on neural network weights for all policies and assign very small initial values to those in the last layer to prevent instability in the feedback controller.

In PPO, value function $\bar{V}$ (see Eq. (4)) is also trained at every iteration when the policy is trained. Even though we desire a decentralized control setting, actor policies are not trained to optimize over local reward maximization, rather they are trained to maximize global (system-wide) reward. For this reason, instead of n value functions ($\bar{V}_i$ for each $i \in \mathcal{C}$), there is a single value function (critic) for all actors combined, and this function's argument is the system-wide state. In contrast, Multi-Agent Deep Deterministic Policy Gradients (MADDPG) [24], a method that also employs multiple policy networks in a policy gradient approach, expects each 'agent' not only to observe and act locally (with its own policy network), but to also have its own critic network (value function estimate) and to estimate the policy network of others. In that paper, each agent is assumed to have their own reward function or objective. However, since there's always a global objective (single reward signal), then only one critic function suffices, and the policy networks train simultaneously to adapt to each other.

Due to the way the policy architecture was chosen, there is still a single policy function π from a standard PPO algorithm perspective, with $2n$ inputs and $2n$ outputs in this formulation.

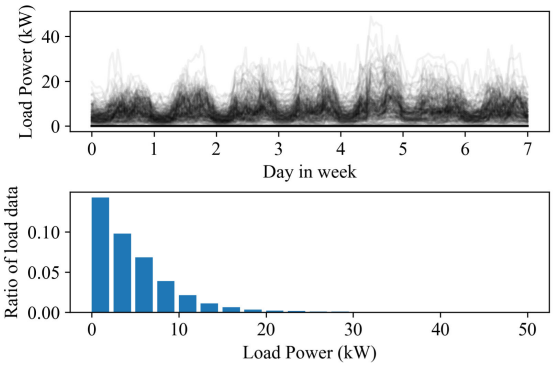


FIGURE 5. **Example of real load data from grid located in the Midwest U.S. [25]. (Top) about 200 load profiles in the first week of January 2017. (Bottom) load distribution over the entire year of 2017.**

Since there is a single value function, the PPO agent is trained as usual, without any explicit changes to the algorithm, other than the aforementioned restriction that the policy network connects input to output only at the same bus. To implement this, the optimizer (e.g. Adam optimizer in PyTorch) is told to ignore the weights (initialized and left at zero) in the policy network which link actions at one bus to states at another.

VI. NUMERICAL SIMULATION

In this section, we apply the proposed policy architecture and use PPO to solve the MDP. Numerical simulations are conducted on a 240-node distribution grid (see Fig. 6) using OpenDSS to solve unbalanced power flow, as described in Section IV. All parameters associated with this network are obtained from real line parameters and real load data, based on an anonymous distribution grid in Midwest U.S. [25]. Experiment details (e.g. software and hardware details) are found in the Appendix I.

Based on numerical simulations, as shown in Fig. 7, we have found that the decentralized agent is more sample efficient and trains with fewer fluctuations and variance in episodic rewards over the learning process. On the other hand, the centralized agent takes a bit less computation time (about 20% less) per iteration, yet takes more iterations to converge.

A. SIMULATION SETUP

The RL agent interacts with the distribution grid every 10 ms (the time step), and each episode contains 100 time steps, for a total of one second per episode. μ is set to 0.1 to favor voltage regulation over solar production maximization. We use the distribution grid shown in Fig. 6, where $N = 240$, and we select $n = 16$ and $n = 194$ for the case studies that follow. 194 is the number of controllable nodes provided originally with the OpenDSS model of this grid. For each of these 194 nodes, we have one year (2017) of real historical load data (P^l, Q^l), which we take advantage of to generate random

Algorithm 1 One Episode of Interacting With the Distribution Grid

function `RunEpisode(in_training, `$\mathcal{D}$`)` ▷ `in_training` is True or False, $\mathcal{D}$ is experience buffer
  Specify load & solar scenario (random & time of day).
  if `in_training` **then**
    Set agent policies to sample actions stochastically
  else
    Set agent policies to sample actions deterministically
  while episode not done **do**
    for each controllable bus i in grid **do**
      Observe local voltage and active power measurements.
      Use policy π_i to sample changes to active & reactive power setpoints at local PV panels.
    Collect rewards and append actions, observations, & rewards to $\mathcal{D}$. ▷ Do this only if `in_training`

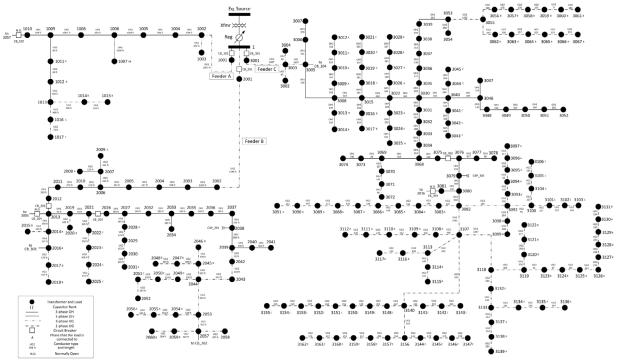


FIGURE 6. Real distribution grid located in the Midwest U.S. [25], with 240 nodes excluding the substation node.

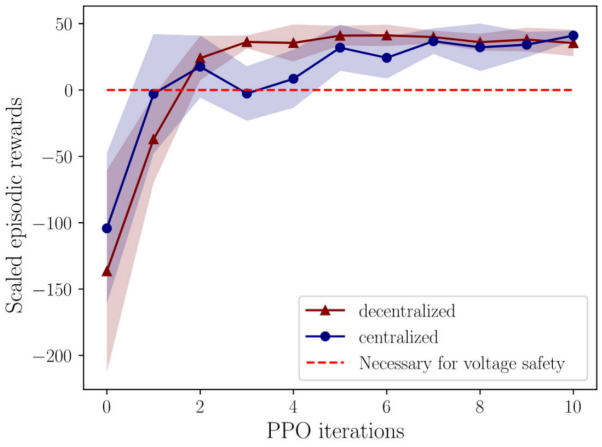


FIGURE 7. RL training curve for centralized vs. decentralized policies. Decentralized agent trains more monotonically and with less variance in episodic rewards. For both, rewards below 0 indicate voltages are not within user-defined safety boundary.

samples for our simulation at the start of every episode. A short summary of the load data is shown in Fig. 5.

Since each episode is 1 second, it is fair to assume that fluctuations in p^{env} and $(P^l,\ Q^l)$ are negligible within one episode. For this reason, at the beginning of each episode, as indicated in Algorithm 1, we randomly generate and fix p^{env} and $(P^l,\ Q^l)$ for the remainder of the episode. Maximum solar power output, p^{env}, for training purposes is randomly selected in each episode as a multiple of the net load at the same bus. For example, at a given bus, if the net load is x kW, then p^{env} at the same bus is chosen uniformly between 0 and $2x$ kW. This helps generate a diverse set of scenarios, ranging from net over-consumption to net over-production at each bus.

The implementation of each episode is detailed in Algorithm 1. If the episode is being run for the purpose of training, experiences are collected and stored in the experience buffer (also known as the replay buffer). Furthermore, each agent acts deterministically during online interaction with the environment. However, during training, actions are sampled from a Gaussian, whose mean and variance are determined by the policy's neural network. The `in_training` boolean (True or False) is used in the algorithm to distinguish between online execution only, or training. Finally, Algorithm 2 describes how interaction with the environment is used to train the policy and value networks ($\pi_i\ \forall i$ and $\bar{V}$).

B. CASE STUDY ON A SMALLER (16-BUS) SUBSYSTEM

In this case study, we compare the use of a centralized policy to that of a decentralized policy, presented in Section V.

Since $n = 16$, neural networks of both centralized and decentralized policies have 32 inputs and 32 outputs. The standard choice of 2 hidden layers with 64 neurons per layer is made for the centralized policy, with $tanh(\cdot)$ activation functions, whereas the decentralized policy splits into 16 sub-policies each with 2 inputs, 2 outputs, and two hidden layers each with 4 neurons. This gives both the centralized and decentralized policies a 'height' of 64 neurons in the hidden layer ($4 \times 16 = 64$), but a total of 8352 parameters to tune for the former and 672 for the latter. In fact, in the decentralized case, we assign 16 different Adam optimizers, one to tune each sub-policy, so it's not so much 672 parameters to optimize per PPO iteration, rather 42 per optimizer, compared to 8352 per (single) optimizer in the centralized setting.

The training curve for each is shown in Fig. 7, where each 'PPO iteration' on the x-axis refers to 2048 steps of interacting with the environment (or 20s, considering 10ms time step). It is evident that the centralized agent does not

Algorithm 2 Training Decentralized PPO for Distribution Grids

1: Initialize the following:
- policy neural network, π_i for each bus i
- value neural network, $\bar{V}$
- PPO hyper-parameters
- experience buffer, $\mathcal{D}$ ▷ memory of actions, observations and rewards used in PPO

2: `in_training` ← True
3: **while** number of episodes less than threshold **do**
4: **for** each episode **do**
5: `RunEpisode(in_training,` $\mathcal{D}$`)` ▷ Defined in Algorithm 1
6: **every** pre-specified number of episodes **do**
7: Use $\mathcal{D}$ to update policy and value networks according to PPO.

out-perform the decentralized agent, and is clearly less interpretable, and requires a wide communication infrastructure to implement in practice. Note: in both centralized and decentralized cases, *value* function V is centralized (fully connected neural network). That is, the RL agent is centralized during training (computer simulation), but decentralized during execution (real-world).

In classic RL benchmarks, a threshold on average cumulative rewards is chosen to determine when the learning problem is *solved*. This helps the person who monitors and debugs the learning process to get a sense of progress to save time and effort. In our context, the threshold is set to 0, as shown in Fig. 7, for the following reasons. We know that $R_V \leq 0$ and $R_P \geq 0$, from Eq. (3g,3f). Both reward terms have been designed in such a way that the magnitudes of the rewards are of order 1 or less during normal operating conditions. Moreover, say that the user desires to keep voltages within $1 \pm \delta$. It is then a fact that $(R_V + \mu R_P) \geq 0$ at every bus only if voltages are kept within the desired region at all buses. It logically follows that if the inequality does not hold, then the voltage at least at one bus must be outside the desired region. Thus, we can state that by simply monitoring the training curve, one may claim that not all voltages are inside $1 \pm \delta$ if the curve is still below the threshold, and that certainly more time needs to be given before ending the training progress. This necessary condition on voltage serves as a useful tool for users who seek to implement this approach. *Note:* in that figure, the term 'voltage safety' merely refers to voltages being inside $1 \pm \delta$. Fig. 7 demonstrates that the decentralized agent permanently crosses this threshold after 2 iterations, while the centralized takes 4 iterations to do so.

By these results, we claim that one can obtain results for a decentralized agent that are similar to, or even better than, those for a centralized agent, simply by manipulating the neural network's architecture.

C. CASE STUDY ON A LARGER (194-BUS) SUBSYSTEM

In the previous subsection, we compared centralized and decentralized policy architectures. In this subsection, we dig deeper to examine our proposed framework from purely a power systems perspective. We ask the following question:

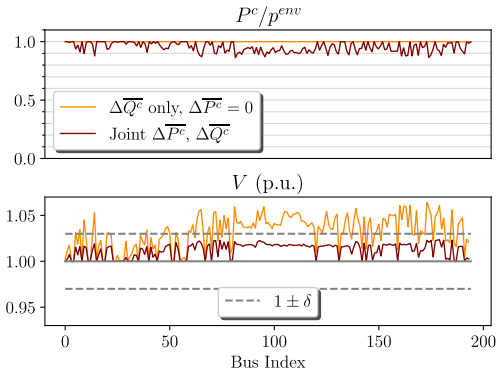


FIGURE 8. Comparison between control policies under a scenario with deep photovoltaic penetration. Joint provision of real and reactive power yields improved voltage profile with minor power sacrifice.

what is the impact of joint real and reactive power control (as opposed to just the latter) on system-wide voltage profile in the midst of deep photovoltaic penetration?

Consider $n = 194$ buses, and the same grid as before, with controllable real and reactive power inverter setpoints. As shown in Fig. 8, when maximum real power is drawn from the solar panels, leaving less reactive power support, deep photovoltaic penetration causes over-voltage. With joint provision of real and reactive power, the RL agent manages to keep voltages within the user-defined desired region ($1 \pm \delta$). Surprisingly, a small reduction in real power injection was needed to achieve this effect. Fig. 9 shows the steady-state distribution of real power consumption per bus, as a ratio to maximum possible injection (p^{env}). It is worth noting how well the voltage was improved system-wide, even though most solar panels produced near maximum output (note the 0.85 on the y-axis of both figures 8 and 9). This justifies the value in considering joint provision of real and reactive power support.

Indeed, there exist plenty of studies in the body of literature that implement some form of power curtailment (to provide reactive power support) for the purposes of voltage regulation. We propose the novelty here is twofold. Firstly, by using

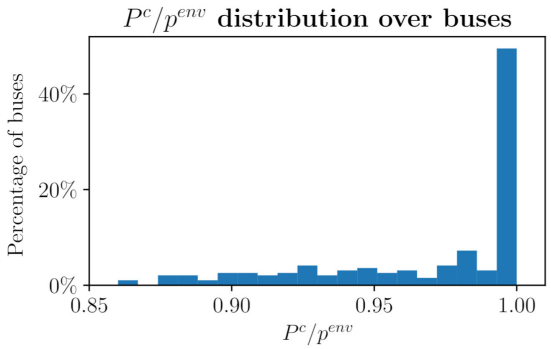


FIGURE 9. Histogram of P^c/p^{env} under RL policy, using results of Fig. 8. Most buses inject near-maximum real power.

RL to automatically learn to balance between voltage regulation and maximum power utilization, the user does not need to design heuristics to reach similar results or to explicitly rely on any understanding of how the system works. Secondly, as shown in Fig. 8, very little power curtailment is required to significantly improve the voltage profile and keep it well within the desired thresholds. This is made possible by the structure of the proposed decentralized PPO policy which allows all the agents to simultaneously train to adapt to one another during exploration, despite the lack of communication between them during real-time operation.

VII. CONCLUDING REMARKS

This paper introduces a reinforcement learning-based voltage control strategy with joint provision of real and reactive power for distribution grids with deep photovoltaic penetration. The joint real power and voltage support problem is formulated as a Markov Decision Process with rewards parametrized to balance between voltage deviation minimization and solar production maximization.

Compared with conventional multi-agent RL algorithms, we develop a tailor-designed *decentralized PPO (Proximal Policy Optimization)* algorithm that would both work well with large continuous action spaces and simplify the optimization process by updating multiple policies simultaneously. We demonstrate that by reducing the search space, for the simper 16-bus subsystem, from over 8000 parameters for a centralized setting to under 700 parameters for a decentralized setting, we still achieve similar, and in some cases better, average rewards. This size reduction is on the order of 10 or so, but for the 194-bus subsystem, it is on the order of 100. Numerical simulations on a 240-node distribution grid based on real parameters show that it is not always the best strategy to absorb all the solar power available. This observation implies that it would benefit the distribution grid the most if some fraction of the real-time power produced from the PV panels can be locally absorbed.

This paper also proposes and verifies a fully decentralized (communication-free) approach for this type of control, which can be implemented on existing physical infrastructure, helping alleviate problems related to communication failure or cyber-attacks. In future work, competition between agents is considered, whereby the inverter at each bus seeks to maximize local, not system-wide, rewards. Further research could also investigate the optimal combination of local energy storage together with PV panels for real-time operation.

APPENDIX I. EXPERIMENT DETAILS

Software: Simulations are conducted in Python 3.7, interfacing with OpenDSS [23] (unbalanced distribution grid simulator) and using PyTorch [26] (Python-based library to train neural networks) to model, build and train actor and critic neural networks. Hardware: Lenovo, 64-bit Windows 10, Intel®Core^{TM}i7-6700HQ CPU @ 2.60Ghz, 16.0 GB RAM.

Proximal policy optimization (PPO) parameters: Number of steps per training update of 512 to 2048 time steps. Considering each episode/scenario involved 100 time steps, this implies on the order of 10 episodes every actor and critic network update. Batch size of 16 time steps with 10 epochs. This means that for each training update, the aforementioned neural networks are updated by performing nonlinear regression 10 times, each using 16 time steps randomly selected from the experience buffer (refer to Algorithm 2).

REFERENCES

[1] M. Farivar, R. Neal, C. Clarke, and S. Low, "Optimal inverter VAR control in distribution systems with high PV penetration," in *Proc. IEEE Power Energy Soc. Gen. Meeting*, Jul. 2012, pp. 1–7.

[2] J. Seuss, M. J. Reno, R. J. Broderick, and S. Grijalva, "Improving distribution network PV hosting capacity via smart inverter reactive power support," in *Proc. IEEE Power Energy Soc. Gen. Meeting*, Jul. 2015, pp. 1–5.

[3] V. Kekatos, G. Wang, A. J. Conejo, and G. B. Giannakis, "Stochastic reactive power management in microgrids with renewables," *IEEE Trans. Power Syst.*, vol. 30, no. 6, pp. 3386–3395, Nov. 2015.

[4] *IEEE Standard for Interconnection and Interoperability of Distributed Energy Resources With Associated Electric Power Systems Interfaces*, IEEE Std 1547-2018, Apr. 2018.

[5] X. Su, M. A. S. Masoum, and P. J. Wolfs, "Optimal PV inverter reactive power control and real power curtailment to improve performance of unbalanced four-wire LV distribution networks," *IEEE Trans. Sustain. Energy*, vol. 5, no. 3, pp. 967–977, Jul. 2014.

[6] B. Zhang, A. Y. S. Lam, A. D. Dominguez-Garcia, and D. Tse, "An optimal and distributed method for voltage regulation in power distribution systems," *IEEE Trans. Power Syst.*, vol. 30, no. 4, pp. 1714–1726, Jul. 2015.

[7] H. Zhu and H. J. Liu, "Fast local voltage control under limited reactive power: Optimality and stability analysis," *IEEE Trans. Power Syst.*, vol. 31, no. 5, pp. 3794–3803, Sep. 2016.

[8] W. Lin, R. Thomas, and E. Bitar, "Real-time voltage regulation in distribution systems via decentralized PV inverter control," in *Proc. 51st Hawaii Int. Conf. Syst. Sci.*, 2018, pp. 2681–2682.

[9] G. Qu and N. Li, "Optimal distributed feedback voltage control under limited reactive power," *IEEE Trans. Power Syst.*, vol. 35, no. 1, pp. 315–331, Jan. 2020.

[10] S. Magnusson, G. Qu, C. Fischione, and N. Li, "Voltage control using limited communication," *IEEE Trans. Control Netw. Syst.*, vol. 6, no. 3, pp. 993–1003, Sep. 2019.

[11] R. El Helou, D. Kalathil, and L. Xie, "Communication-free voltage regulation in distribution networks with deep PV penetration," in *Proc. 53rd Hawaii Int. Conf. Syst. Sci.*, 2020, pp. 3205–3206.

[12] H. Xu, A. D. Dominguez-Garcia, and P. W. Sauer, "Optimal tap setting of voltage regulation transformers using batch reinforcement learning," *IEEE Trans. Power Syst.*, vol. 35, no. 3, pp. 1990–2001, May 2020.

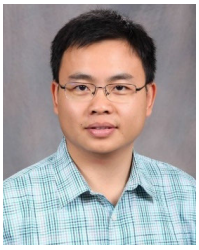
[13] Y. Xu, W. Zhang, W. Liu, and F. Ferrese, "Multiagent-based reinforcement learning for optimal reactive power dispatch," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 42, no. 6, pp. 1742–1751, Nov. 2012.
[14] Q. Yang, G. Wang, A. Sadeghi, G. B. Giannakis, and J. Sun, "Two-timescale voltage control in distribution grids using deep reinforcement learning," *IEEE Trans. Smart Grid*, vol. 11, no. 3, pp. 2313–2323, May 2020.
[15] B. L. Thayer and T. J. Overbye, "Deep reinforcement learning for electric transmission voltage control," 2020, *arXiv:2006.06728*. [Online]. Available: http://arxiv.org/abs/2006.06728
[16] W. Wang, N. Yu, Y. Gao, and J. Shi, "Safe off-policy deep reinforcement learning algorithm for volt-VAR control in power distribution systems," *IEEE Trans. Smart Grid*, vol. 11, no. 4, pp. 3008–3018, Jul. 2020.
[17] C. Li, C. Jin, and R. Sharma, "Coordination of PV smart inverters using deep reinforcement learning for grid voltage regulation," in *Proc. 18th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2019, pp. 1930–1937.
[18] W. Wang, N. Yu, J. Shi, and Y. Gao, "Volt-VAR control in power distribution systems with deep reinforcement learning," in *Proc. IEEE Int. Conf. Commun., Control, Comput. Technol. Smart Grids (SmartGridComm)*, Oct. 2019, pp. 1–7.
[19] F. Katiraei and M. R. Iravani, "Power management strategies for a microgrid with multiple distributed generation units," *IEEE Trans. Power Syst.*, vol. 21, no. 4, pp. 1821–1831, Nov. 2006.
[20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*. [Online]. Available: http://arxiv.org/abs/1707.06347
[21] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 1998.
[22] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1889–1897.
[23] R. C. Dugan and D. Montenegro, "The open distribution system simulator opendss), reference guide," Electr. Power Res. Inst. (EPRI), Tech. Rep., 2018. [Online]. Available: https://spinengenharia.com.br/wp-content/uploads/2019/01/OpenDSSManual.pdf
[24] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," 2017, *arXiv:1706.02275*. [Online]. Available: http://arxiv.org/abs/1706.02275
[25] F. Bu, Y. Yuan, Z. Wang, K. Dehghanpour, and A. Kimber, "A time-series distribution test system based on real utility data," in *Proc. North Amer. Power Symp. (NAPS)*, Oct. 2019, pp. 1–6.
[26] A. Paszke *et al.*, "Automatic differentiation in pytorch," Tech. Rep., 2017. [Online]. Available: https://openreview.net/forum?id=BJJsrmfCZ



RAYAN EL HELOU (Student Member, IEEE) received the B.S. and M.S. degrees in electrical and computer engineering from The Ohio State University, Columbus, OH, USA. He is currently pursuing the Ph.D. degree with the Department of Electrical and Computer Engineering, Texas A&M University, College Station, TX, USA. His research interests include power systems dynamics, modeling and control of smart grids, and reinforcement learning



DILEEP KALATHIL (Senior Member, IEEE) received the M.Tech. degree from IIT Madras and the Ph.D. degree from the University of Southern California (USC) in 2014. From 2014 to 2017, he was a Post-Doctoral Researcher with the Department of Electrical Engineering and Computer Sciences, University of California at Berkeley, Berkeley. He is currently an Assistant Professor with the Department of Electrical and Computer Engineering, Texas A&M University, College Station, TX, USA. His research interests include intelligent transportation systems, sustainable energy systems, data driven optimization, online learning, stochastic control, and game theory. He was a recipient of the Best Academic Performance from the EE Department, IIT Madras and the Best Ph.D. Dissertation Prize in the USC Department of Electrical Engineering.



LE XIE (Senior Member, IEEE) received the B.E. degree in electrical engineering from Tsinghua University, Beijing, China, in 2004, the M.S. degree in engineering sciences from Harvard University, Cambridge, MA, USA, in 2005, and the Ph.D. degree from the Department of Electrical and Computer Engineering, Carnegie Mellon University, Pittsburgh, PA, USA, in 2009. He is currently a Professor with the Department of Electrical and Computer Engineering, Texas A&M University, College Station, TX, USA. His research interests include modeling and control of large-scale complex systems, smart grids application with renewable energy resources, and electricity markets